

Manuel NSI Première - livret méthodes

Édition de travail 2026-2027

Table des matières

1	Types construits : p-uplets, tableaux, dictionnaires
---	--

1 Types construits : p-uplets, tableaux, dictionnaires

MÉTHODE M1 — CHOISIR ENTRE TUPLE ET LISTE

Quand l'utiliser? Quand tu dois stocker plusieurs valeurs ensemble et que tu hésites entre un tuple et une liste.

Pas à pas :

1. Demande-toi si les données seront modifiées après création (ajout, suppression, remplacement).
2. Si **non** (données fixes) : utilise un **tuple**. Exemples : coordonnées d'un point, date de naissance, résultat renvoyé par une fonction.
3. Si **oui** (données évolutives) : utilise une **liste**. Exemples : liste de scores en cours de partie, panier d'achats.
4. En cas de doute, pose-toi la question : « cette collection va-t-elle grandir ou changer ? »

Exemple :

```
1 # Données fixes : on choisit un tuple
2 position = (3, 7)
3 couleur_rgb = (255, 128, 0)
4
5 # Données évolutives : on choisit une liste
6 scores = [12, 8, 15]
7 scores.append(20) # on ajoute un score
```

Pièges :

- Utiliser une liste pour des données qui ne changent jamais : cela fonctionne, mais un tuple exprime mieux l'intention et protège contre les modifications accidentelles.
- Tenter de modifier un tuple avec `t[0] = 5` provoque une `TypeError`.

Vérifier : ton choix est cohérent si tu n'as jamais besoin de `append`, `remove` ou d'affectation par indice sur un tuple.

S'entraîner : exercices C1.

MÉTHODE M2 — CONSTRUIRE UN TABLEAU EN COMPRÉHENSION

Quand l'utiliser? Quand tu veux créer une liste à partir d'une règle de calcul ou d'un filtre, en une seule ligne.

Pas à pas :

1. Identifie la **source** : sur quoi itères-tu ? (un range, une liste existante, etc.)
2. Identifie l'**expression** : que calcules-tu pour chaque élément ?
3. (Optionnel) Identifie la **condition** : quels éléments gardes-tu ?
4. Écris la compréhension selon le patron : `[expr for var in source if cond]`.

Exemple :

```
1 # Carres des entiers de 0 à 9
2 carres = [x**2 for x in range(10)]
3 # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```

4
5 # Garder uniquement les nombres pairs d'une liste
6 nombres = [3, 8, 12, 5, 20, 7]
7 pairs = [n for n in nombres if n % 2 == 0]
8 # [8, 12, 20]

```

Pièges :

- ▶ Oublier les crochets [...] : sans eux, tu obtiens un générateur, pas une liste.
- ▶ Placer le if avant le for : la condition de filtrage se place *après* le for.
- ▶ Écrire une compréhension trop complexe : si tu as besoin de plus de deux lignes de logique, préfère une boucle classique.

Vérifier : affiche la liste obtenue avec print et contrôle sa longueur avec len.

S'entraîner : exercices C2.

MÉTHODE M3 — PARCOURIR UNE GRILLE LIGNE PAR LIGNE

Quand l'utiliser ? Quand tu travailles avec un tableau de tableaux (grille, image, plateau de jeu) et que tu dois accéder à chaque case.

Pas à pas :

1. Repère le nombre de lignes : nb_lignes = len(grille).
2. Repère le nombre de colonnes : nb_colonnes = len(grille[0]).
3. Écris une première boucle for i in range(nb_lignes) pour parcourir les lignes.
4. Écris une seconde boucle imbriquée for j in range(nb_colonnes) pour parcourir les colonnes.
5. Accède à la case courante avec grille[i][j].

Exemple :

```

1 grille = [
2     [1, 2, 3],
3     [4, 5, 6],
4 ]
5
6 for i in range(len(grille)):
7     for j in range(len(grille[0])):
8         print(grille[i][j], end=" ")
9     print() # retour a la ligne apres chaque ligne
10 # Affiche :
11 # 1 2 3
12 # 4 5 6

```

Pièges :

- ▶ Confondre lignes et colonnes : grille[i] est la ligne i ; grille[i][j] est la case à la ligne i, colonne j.
- ▶ Inverser les indices : grille[j][i] au lieu de grille[i][j] donne un résultat faux (et parfois une erreur d'indice).
- ▶ Supposer que toutes les lignes ont la même longueur sans le vérifier.

Vérifier : affiche i, j, grille[i][j] dans la boucle pour contrôler que chaque case est bien visitée.

S'entraîner : exercices C3.

MÉTHODE M4 — CHERCHER DANS UN DICTIONNAIRE SANS ERREUR

Quand l'utiliser ? Quand tu veux accéder à une valeur dans un dictionnaire sans risquer une `KeyError` si la clé est absente.

Pas à pas :

1. **Méthode 1 — tester avec `in`** : vérifie si la clé existe avant d'y accéder.
2. **Méthode 2 — utiliser `.get()`** : appelle `dico.get(cle, valeur_par_defaut)` qui renvoie la valeur par défaut si la clé est absente (au lieu de provoquer une erreur).
3. Choisis la méthode adaptée : `in` quand tu dois exécuter un bloc particulier selon la présence de la clé ; `.get()` quand tu veux simplement récupérer une valeur avec un repli.

Exemple :

```
1 inventaire = {"pommes": 5, "bananes": 3}
2
3 # Methode 1 : tester avec in
4 if "cerises" in inventaire:
5     print(inventaire["cerises"])
6 else:
7     print("Pas de cerises en stock")
8
9 # Methode 2 : utiliser .get()
10 qte = inventaire.get("cerises", 0)
11 print(qte) # 0 (valeur par default)
```

Pièges :

- ▶ Accéder directement avec `dico[cle]` sans vérification : si la clé est absente, Python lève une `KeyError`.
- ▶ Oublier le second argument de `.get()` : sans valeur par défaut, `.get()` renvoie `None` (pas une erreur, mais potentiellement inattendu).
- ▶ Confondre `in` sur un dictionnaire (cherche parmi les *clés*) et `in` sur une liste (cherche parmi les *éléments*).

Vérifier : teste ton code avec une clé présente *et* une clé absente pour confirmer qu'aucune erreur ne se produit.

S'entraîner : exercices C4.

MÉTHODE M5 — COPIER UNE LISTE SANS ALIAS

Quand l'utiliser ? Quand tu veux créer une copie indépendante d'une liste, de sorte que modifier la copie ne modifie pas l'original.

Pas à pas :

1. Identifie le problème : copie = originale ne crée **pas** une copie, mais un **alias** (les deux noms désignent le même objet en mémoire).
2. Pour obtenir une vraie copie, utilise l'une de ces méthodes :
 - ▶ `copie = list(originale)`
 - ▶ `copie = originale[:]` (tranche complète)
3. Vérifie que les deux listes sont bien distinctes en modifiant la copie et en affichant l'original.

Exemple :

```
1 originale = [1, 2, 3]
2
3 # Alias : les deux variables pointent vers le meme objet
4 alias = originale
5 alias.append(4)
6 print(originale) # [1, 2, 3, 4] -- modifiee aussi !
```

```
7
8 # Copie independante
9 originale = [1, 2, 3]
10 copie = list(originale)
11 copie.append(4)
12 print(originale) # [1, 2, 3] -- inchangee
13 print(copie)     # [1, 2, 3, 4]
```

Pièges :

- ▶ Croire que = copie une liste : c'est l'erreur la plus fréquente. L'affectation crée un alias, pas une copie.
- ▶ Attention aux listes imbriquées : `list()` et `[:]` ne font qu'une copie *superficielle*. Si la liste contient d'autres listes, les sous-listes restent partagées.

Vérifier : après la copie, teste copie is originale. Si le résultat est False, tu as bien deux objets distincts.

S'entraîner : exercices C5.